

2026 项目

1. Bootloader

主要执行启动加载，完成从Flash将APP加载到RAM中运行

1. STM32启动过程

- (1) 根据boot引脚选择启动模式，从主闪存存储器启动
- (2) 从启动处获取**设置栈顶地址**
- (3) 在下一个地址获取并初始化PC的值指向复位向量，**跳转Reset_Handler函数地址执行**
- (4) 在**Reset_Handler**跳转运行SystemInit，进行时钟初始化和设置异常向量表地址（SCB->VTOR）
- (5) 调用__main，标准C库函数，执行一系列操作后，最终调用main函数

2. 应用程序启动过程

- (1) 读app.bin的头部文件，得到应用程序的加载地址、应用程序的大小
- (2) 从flash中复制APP到加载地址（源地址写死）
- (3) 设置SP指向向量表起始地址（因为已经作了向量跳转），PC指向下一个地址Reset_Handler，跳转执行app

2. OTA升级

主要通过串口实现了app程序的升级

1. libzmodem移植工作：

- (1)**对libzmodem进行裁剪**，虚拟机作为接收方，用串口终端向虚拟机发送app固件

注释检验码校验、错误检测处理、不同情况实现同一文件操作等不影响正常使用的代码，同时测试注释后代码仍可正常运行

- (2)**去掉了malloc、free等耗时函数，改为固定长度**（文件名）

- (3)**数据收发：改为串口操作**

使用UART接收中断，把数据存入环形buffer

write（本意是写数据到控制台），改为使用UART发送数据（putstr）

zreadline（本意是读取控制台得到数据），改为读UART得到数据

getchar从环形buffer中得到数据（直接读串口会太快，没有办法及时接收）

(4)文件操作：改为内存操作

fopen：确定数据保存内存地址（实现头部信息解析）

fwrite/fputc：写内存

获得文件状态函数改为返回文件大小

指定变量位置：u8 receive_buff[buff_size] **_attribute_**((at(0X20004000)));

(5)将固件拷贝到flash中的应用程序地址

3. ZMODEM协议

ZMODEM：有**流式传输**、**自动开始**、**断点续传**等优势，被广泛用于串口文件传输，适用于固件升级、配置传输等

1.等待传输初始化

程序首先调用 `zm_await("rz\r", ...)` 等待对端开始 rz 传输过程

2. 接收 ZMODEM 帧头

主循环中调用 `zm_await_header()` 获取协议帧头并解析：

3.文件头（ZFILE）处理

当接收到 `ZFILE` 时，调用 `zm_read_data_block()` 获取文件名，并打开写入文件

4.数据帧（ZDATA）接收

进入 `ZDATA` 状态后，程序进入子循环读取数据块，识别**结束符号**（如 `CRCE`、`CRCG`、`CRCQ`、`CRCW`）：

5.异常与错误处理

如果 CRC 校验失败或其他错误，通过 `ZRPOS` 回应远程重发

4. 循环buf

循环缓冲区（环形缓冲区/循环队列）：管理固定大小的数据存储空间，通过将缓冲区的末尾连接到起始位置形成一个闭合的环，来实现数据的循环读取

1.结构 缓冲区空间

写指针：指向下一个可写入数据的位置

读指针：指向下一个可以读取数据的位置

2.操作

（1）初始化：读写指针指向缓冲区的开始位置，即索引0

（2）写入数据：在写指针指向的位置写入数据，如果写指针达到缓冲区的末端，它会绕回开头

缓冲区已满：写指针追上读指针

覆盖模式/阻塞模式

(3) 读取数据：在读指针指向的位置读取数据，读指针前移，如果读指针达到缓冲区的末端，它会绕回开头

空缓冲区：读指针和写指针相同

阻塞模式/非阻塞模式

5. 内存分布管理

1.map文件：有不同数据段的信息统计，可以看到SRAM的分配

Code+RO-data：代码段，只读段

RW-data：已初始化的读写数据（初始化的全局变量和静态变量）

ZI-data：未初始化的读写数据（定义了但没有初始化的全局变量和静态变量）

堆栈的大小是程序开始运行后才能确定

2.flash： bootloader（起始0x8000 0000，32k）、APP（固定0x8010 0000,包括头部+Code+RO-data+RW-data，约40k）

3.ram:数据段（起始0x2000 0000 RW-data + ZI-data）、堆、栈（0x8010 0000）、用户RAM（加载位置可以生成指定，首地址为栈地址，程序下载和加载区，预计100k）

6. flash操作

指定地址写操作：判断地址是否有效（只能操作APP程序部分） -> 解锁 -> 擦除 -> 写入数据->上锁 -> 读验证

12个扇区，bootloader 0，1扇区（16KB），APP2，3扇区（16KB）

擦除以扇区为单位，每次写入一个32位地址数据

7. 关键汇编代码

1.汇编指令

(1) ldr：从存储器中加载一个字到一个寄存器中

(2) str：把一个寄存器按字存储到存储器中

(3) bx：转移到由寄存器给出的地址

2.源码

```
SVC_Handler PROC
```

```
EXPORT SVC_Handler
```

```
ldr r0, =(0x20000000 + 0x10000 - 4)
```

```
ldr r0, [r0] ; r0 = new_vector  
ldr r0, [r0, #0x2c] ; new SVC_Handler  
bx r0
```

ENDP

触发SVC中断->加载app的向量表->跳转相应中断服务函数

8. 项目中的问题

1. 问题描述

应用程序启动失败，执行完bootloader后卡死

2. 原因排查

- (1) 通过串口打印调试发现，bootloader成功跳转app，但在SVC中断函数时崩溃
- (2) 利用IDE的单步调试功能，发现SVC中断函数返回时LR异常归0
- (3) 查看反汇编文件，发现BLX LR破坏了LR中的特殊值
- (4) 原先代码中触发中断采用bootloader中断跳转app中断，但是使用c语言编写的跳转代码，编译器生成的汇编代码不可用

3. 问题解决

将bootloader中断中的C程序跳转代码改为汇编跳转代码，利用LDR指令读取固定在app程序头的向量表，跳转执行对应函数，app成功启动

4. 其他：在内存中进行调试

为什么要在内存中进行调试：bootloader会将flash中的应用程序拷贝到内存中进行跳转运行

通过创建新的调试配置，更换调试算法内存地址、调试基地址改为内存基地址（JTAG调试原理：加载一段调试算法代码到内存），创建初始化文件（编写一个ram.ini文件，代替硬件初始化操作：设置SP，PC，Vector等），放入工程目录，在Debug选项卡中使用初始化文件

1. U-Boot移植

1. 修改文件及文件夹名称

- (1) 开发板对应的头文件

mx6ull_alientek_emmc.h里面有很多宏定义，这些宏定义基本用于配置uboot，也有一些I.MX6ULL的配置项目

使能或者禁止uboot的某些功能，那就在mx6ull_alientek_emmc.h里面做修改即可

- (2) 开发板对应的板级文件夹

uboot中每个板子都有一个对应的文件夹来存放板级文件，比如开发板上外设驱动文件等等

(3) U-Boot 图形界面配置文件

2.新建mx6ull_alientek_emmc.sh 的 shell 编译脚本

使用 imxdownload 将新编译出来的 u-boot.bin 烧写到 SD 卡中测试

3.修改LCD驱动

- ①、LCD 所使用的 GPIO，查看 uboot 中 LCD 的 IO 配置是否正确。
- ②、LCD 背光引脚 GPIO 的配置。
- ③、LCD 配置参数是否正确。

像素时钟pixclock的含义不同

$\text{pixclock} = (1/51200000) * 10^12 = 19531$

设置panel为TFT7016

在 uboot 中修改panel 的值为TFT7016

4.修改网络驱动

底板网络芯片更换为SR8201F，网络PHY地址也有改变

I.MX6UL/ULL 内部有个以太网 MAC 外设，也就是ENET，需要外接一个PHY芯片来实现网络通信功能，也就是**内部MAC+外部PHY**芯片的方案（优势速度快）

①、ENET1复位引脚初始化

②、SR8201F的器件ID

③、SR8201F驱动

(1) 修改uboot中的ENET1和ENET2的PHY地址和驱动

(2) 删除uboot中74LV595的驱动代码

(3) 添加I.MX6U-ALPHA开发板网络复位引脚驱动

2. 启动环境配置

Linux系统选择通过TFTP从网络启动，并且使用NFS挂载网络根文件系统

1.设置uboot中的环境变量

(1) **bootcmd**

保存着uboot默认启动命令，uboot倒计时结束以后就会执行bootcmd中的命令

比如读取EMMC或者NAND Flash中的Linux内核镜像文件和设备树文件到DRAM中，然后启动Linux内核

修改默认值

tftp 80800000 zImage;tftp 83000000 imx6ull-alientek-emmc.dtb;bootz 80800000 - 83000000

(2) bootargs

保存着 uboot 传递给 Linux 内核的参数

bootargs 环境变量是由mmcargs设置的，mmcargs环境变量

```
console=ttymxc0,115200 root=/dev/nfs rw nfsroot=192.168.1.250:/home/ljg/linux/nfs/
rootfs ip=192.168.1.251:192.168.1.250:192.168.1.1:255.255.255.0::eth0:off
```

展开之后：mmcargs=setenv bootargs console= ttymxc0, 115200 root= /dev/mmcblk1p2 rootwait rw

(1) console：设置linux 终端(或者叫控制台)

设置console为ttymxc0，因为linux 启动以后l.MX6ULL的串口1在linux下的设备文件就是/dev/ttymxc0，并且串口波特率设置为115200

(2) root：设置根文件系统的位置

root=/dev/mmcblk1p2 用于指明根文件系统存放在 mmcblk1 设备的分区 2 中

root 后面有“rootwait rw”， rootwait 表示等待 mmc 设备初始化完成以后再挂载，否则的话 mmc 设备还没初始化完成就挂载根文件系统会出错的。rw表示根文件系统是可以读写的，不加 rw 的话可能无法在根文件系统中进行写操作，只能进行读操作

(3) rootfstype：配置 root，指定根文件系统类型

如果根文件系统为 ext 格式的话此选项无所谓。如果根文件系统是yaffs、jffs或ubifs的话就需要设置此选项，指定根文件系统的类型

3.创建存放驱动模块的目录/lib/modules/4.1.15

使用 modprobe 命令加载驱动模块的时候，驱动模块要存放在此目录下

4.配置busybox，使能depmod命令，然后重新编译busybox

每次启动时接输入 depmod 命令即可自动生成 modules.dep

输入“depmod”命令以后会自动生成modules.alias、 modules.symbols 和 modules.dep 这三个文件

3. Linux启动方式

1.从 EMMC启动Linux系统

将编译出来的Linux 镜像文件zImage 和设备树文件保存在EMMC 中，uboot 从EMMC中读取这两个文件并启动

 同步块 [该内容不支持导出查看]

2.从网络启动 Linux系统

从网络启动linux 系统的唯一目的就是为了调试

将linux镜像文件和根文件系统都 放到Ubuntu 下某个指定的文件夹中

Linux系统选择通过TFTP从网络启动，并且使用NFS挂载网络根文件系统

```
setenv bootargs 'console=ttyMXC0,115200 root=/dev/mmcblk1p2 rootwait rw'
```

```
setenv bootcmd 'tftp 80800000 zImage; tftp 83000000 imx6ull-alientek-emmc.dtb; bootz  
80800000 - 83000000' saveenv
```

4. Linux内核移植

1.根文件系统缺失错误

Linux 内核启动以后是需要根文件系统的，根文件系统存在哪里是由uboot的bootargs环境 变量指定，bootargs 会传递给 Linux 内核作为命令行参数

2.修改名称

3.CPU主频修改

在图形化配置界面，打开默认调频策略选择界面，选择 “performance”

4.修改EMMC驱动

(1) 使能8线EMMC驱动

直接修改设备树

(2) 关闭EMMC 1.8V供电选项

EMMC工作电压是3.3V的，在usdhc2设备树节点中添加 “no-1-8-v” 选项，也就是关闭1.8V这个功能选项

5.网络驱动修改

(1) 修改SR8201F的复位以及网络时钟引脚驱动

(2) 修改fec1和fec2节点的pinctrl-0属性

(3) 修改SR8201F的PHY地址

5. 根文件系统构建

1.根文件系统

根文件系统并不是FATFS这样的文件系统代码，EXT4这样的文件系统代码属于Linux内核的一部分。

Linux 中的根文件系统更像是一个文件夹或者叫做目录，在这个目录里面会有很多的子目录。根目录下和子目录中会有很多的文件，这些文件是Linux运行所必须的，比如**库、常用的软件和命令、设备文件、配置文件**等等。

根文件系统是Linux内核启动以后挂载(mount)的第一个文件系统，然后从根文件系统中读取**初始化脚本**，比如rcS，inittab等。根文件系统和Linux内核是分开的，单独的Linux内核是没法正常工作的，必须要搭配根文件系统

2.BusyBox

BusyBox是一个集成了大量的Linux命令和工具的软件，像ls、mv、ifconfig等命令BusyBox都会提供。BusyBox就是一个大的工具箱，这个工具箱里面集成了Linux的许多工具和命令。

3.编译 BusyBox 构建根文件系统

- (1) 修改Makefile，添加编译器
- (2) busybox中文字符支持
- (3) 配置busybox
- (4) 编译busybox

4.向根文件系统添加lib库

- (1) 向rootfs的“/lib”目录添加库文件
- (2) 向rootfs的“usr/lib”目录添加库文件
- (3) 创建其他文件夹

5.完善根文件系统

- (1) 创建/etc/init.d/rcS 文件

rcS是个shell脚本，Linux内核启动以后需要启动一些服务，而rcS就是规定启动哪些文件的脚本文件

- (2) 创建/etc/fstab 文件

fstab在Linux开机以后自动配置哪些需要自动挂载的分区

- (3) 创建/etc/inittab 文件

init程序会读取/etc/inittab这个文件，inittab由若干条指令组成